

Authentication Flow

Roles

There are four roles in the system. Three are self-registerable; Admin is grant-only.

Role	Self-register	Verification required
Customer	Yes	No — active immediately
Merchant	Yes	Yes — admin must approve
Charity	Yes	Yes — admin must approve
Admin	No — granted by existing admin	—

1. Registration — POST /auth/register

Request body

```
{
  "name": "Youssef Wagih",
  "email": "youssef@example.com",
  "password": "Secure123",
  "phoneNumber": "01022024281",
  "role": "Customer", // Customer | Merchant | Charity
  "businessName": "My Store", // required for Merchant and Charity
```

```
"businessCategory": "Restaurant", // optional, Merchant
"language": "en"                  // "en" | "ar", default "en"
}
```

What happens server-side

1. Role is validated — only `Customer`, `Merchant`, `Charity` are self-registerable.
2. `businessName` is required when role is `Merchant` or `Charity`.
3. Email uniqueness is checked via ASP.NET Core Identity.
4. Phone number uniqueness is checked across all existing users.
5. An `ApplicationUser` is created with the given details.
6. The user is assigned their role via ASP.NET Core Identity's role system.
7. For **Merchant** and **Charity**: a draft `Store` record is created in the same DB transaction, with `VerificationStatus = Unverified`. If user creation succeeds but the store insert fails, the entire transaction rolls back.
8. A welcome email is sent via `IService` (currently a dev stub that logs to console).

Response — Customer (immediately active)

```
{
  "data": {
    "user": { "id": "...", "email": "...", "status": "Act
    "accessToken": "eyJ...",
    "refreshToken": "abc123...",
    "accessTokenExpiresAt": "2026-07-27T11:05:01Z"
  }
}
```

Response — Merchant / Charity (pending verification)

```
{
  "data": {
    "user": { "id": "...", "email": "...", "status": "Pen
    "accessToken": "",
    "refreshToken": "",
  }
}
```

```
    "accessTokenExpiresAt": "0001-01-01T00:00:00Z"  
  }  
}
```

Empty tokens signal to the frontend that the account cannot access protected endpoints yet.

2. Login — POST /auth/login

Request body

```
{ "email": "youssef@example.com", "password": "Secure123"
```

Server-side checks (in order)

1. User looked up by email. Unknown email → generic error (no leak).
 2. If status is `Suspended` or `Banned` → error.
 3. Password verified via Identity's `CheckPasswordAsync`.
 4. If status is `PendingVerification` → returns user data with **empty tokens**. The frontend redirects to the pending-verification screen.
 5. Otherwise → tokens issued (see Token Issuance below).
-

3. Token Issuance

Handled by `AuthTokenIssuer`, called from Register (Customer only), Login, and Refresh.

Access token (JWT)

- Algorithm: `HS256`
- Claims: `sub` (user ID), `email`, `jti` (unique token ID),

`ClaimTypes.NameIdentifier`, `ClaimTypes.Role` (one per role)

- Expiry: 15 minutes (configurable via `Jwt:AccessTokenExpirationMinutes`)

Refresh token

- 64 random bytes, Base64-encoded.
 - Stored in the `RefreshTokens` table with `UserId`, `ExpiresAt`, and the issuing IP address.
 - Expiry: 30 days (configurable via `Jwt:RefreshTokenExpirationDays`).
 - Single-use: each use revokes the old token and creates a new one (rotation).
-

4. Token Refresh — POST /auth/refresh

Request body

```
{ "refreshToken": "abc123..." }
```

Server-side steps

1. Token is looked up in the `RefreshTokens` table.
 2. If not found → error.
 3. If not active (expired or revoked):
 - If the token was previously revoked → **reuse detection**: all active sessions for this user are immediately revoked. Warning logged. The user must log in again.
 - Error returned.
 4. If user is `Suspended` or `Banned` → error.
 5. Old token is stamped `RevokedAt`, `RevokedByIp`, and `ReplacedByToken`.
 6. New token pair issued and persisted.
-

5. Logout — POST /auth/logout

Request body

```
{ "refreshToken": "abc123..." }
```

Revokes the refresh token by setting `RevokedAt`. The access token cannot be actively invalidated (it is stateless and short-lived). After logout the client must discard both tokens.

6. Forgot Password — POST /auth/forgot-password

Request body

```
{ "email": "youssef@example.com" }
```

1. User is looked up silently (always returns 200 to avoid account enumeration).
2. ASP.NET Core Identity generates a password reset token.
3. Token is sent to the user's email via `IEmailService`.
4. In development (when `IEmailService.IsDevStub == true`), the token is returned directly in the response body as `debugToken` so it can be passed straight to reset-password without server-log access.

Response

```
{
  "data": {
    "debugToken": "CfDJ8K..." // null in production
  }
}
```

7. Reset Password — POST /auth/reset-password

Request body

```
{
  "email": "youssef@example.com",
  "token": "CfDJ8K...",
  "newPassword": "NewSecure456"
}
```

1. User looked up. Unknown email → error (token is not leaked).
 2. `userManager.ResetPasswordAsync` verifies the token and sets the new password.
 3. All existing refresh tokens for the user are revoked — forcing re-login on all devices.
-

8. Resend Verification — POST /auth/resend-verification

Request body

```
{ "email": "merchant@example.com" }
```

Re-sends the welcome email for accounts still in `PendingVerification` status. Always returns 200 to avoid enumeration. Used by the email verification screen's "Resend Email" button.

9. Business Onboarding Flow (Merchant /

Charity)

After registration the frontend guides the user through 3 steps. Steps 2 and 3 require the token from registration (empty for pending accounts — the frontend stores the email and uses it directly).

```
Step 1: POST /auth/register (role = Merchant | Charity)
        → Draft store created, account status = PendingVe

Step 2a: PATCH /stores/me/location    (requires Merchant |
        → Sets governorate, city, neighborhood, street,

Step 2b: POST /stores/me/documents    (NO auth required –
        → Uploads CommercialRegistration | TaxIdCertific
        → When all 3 are uploaded: store.VerificationSta

Step 3: GET /stores/me
        → Returns current verification status (Unverified

Admin review:
  GET    /admin/stores/pending        → Lists all pending
  GET    /admin/stores/{id}          → Full store + docu
  PATCH  /admin/stores/{id}/verify   → Approve or Reject
        → Approved: store.VerificationStatus = Verified,
        → Rejected: store.VerificationStatus = Rejected,
```

10. Security Details

Concern	Implementation
Password hashing	ASP.NET Core Identity (PBKDF2)
Access token	JWT HS256, 15 min expiry
Refresh token	64-byte random, 30-day expiry, single-use rotation

Reuse detection	Reused revoked token → all sessions for user revoked
Account lockout	5 failed attempts → 15-minute lockout (Identity default)
Unique email	Enforced by Identity (RequireUniqueEmail = true)
Unique phone	Checked at registration via direct DB query
Token invalidation on password reset	All refresh tokens revoked after reset
Admin self-registration	Blocked — Admin role is grant-only